

CandidateIQ - Methodology (India Runs Track 1)

Team: CandidateIQ / SignalHire

Track: Intelligent Candidate Discovery (Redrob ? Hack2Skill)

Dataset: 100,000 anonymized Redrob candidate profiles (candidates.jsonl)

1. Problem Framing

Recruiters searching a 100k+ talent pool face two failures of keyword search:

1. Vocabulary mismatch - A strong "ML Engineer" may never mention "embeddings" or "FAISS" explicitly but demonstrates equivalent work in summaries and project history.
2. Signal blindness - Keyword systems ignore career trajectory, platform activity, intent to move, and education tier - signals Redrob already captures in redrob_signals.

CandidateIQ ranks candidates for a job description using a 6-signal hybrid model: dense semantic retrieval (FAISS + sentence-transformers) followed by interpretable re-ranking. Every ranked row includes a human-readable reasoning field for explainability.

Target JD (Redrob challenge): Senior AI Engineer - Founding Team (5-9 years, embeddings/retrieval/ranking/LLMs, hybrid Pune/Noida).

2. Architecture

[diagram omitted - see methodology.md]

Pipeline stages:

Stage	Component	Purpose
-------	-----------	---------

Ingest	DocumentParser	Parse JSONL -> CandidateProfile (skills, career, Redrob signals)
--------	----------------	--

Enrich	CareerSignalExtractor	Trajectory, implied seniority, consulting-firm penalty
--------	-----------------------	--

Embed	all-MiniLM-L6-v2 (384-dim)	Shared embedding space for JD + candidates
-------	----------------------------	--

Retrieve	FAISS IndexFlatIP	Cosine similarity over 100k vectors; fetch top_n ? 5 for re-rank headroom
----------	-------------------	---

Score	SignalHireScorer	Weighted 6-signal fusion + hard filters
-------	------------------	---

Explain	_explain()	Per-candidate reasoning string for submission
---------	------------	---

3. Six-Signal Scoring Formula

Default weights (configurable via API / submission script):

Signal | Weight | What it measures

Semantic | 0.40 | Embedding cosine similarity (JD ? candidate profile text)

Career | 0.20 | Years-of-experience fit vs JD range; title progression; consulting penalty

Skill | 0.20 | Overlap with JD skill vocabulary + proficiency depth + implied skills

Activity | 0.10 | Profile completeness, recency, response rate, GitHub score

Intent | 0.05 | open_to_work_flag, applications, recruiter saves

Education | 0.05 | Institution tier + field relevance

Final score:

[diagram omitted - see methodology.md]

Hard filters (optional): minimum/maximum experience, role title filters, required skills.

Why this beats keyword matching:

- Semantic retrieval surfaces candidates whose meaning aligns with the JD even when exact terms differ (e.g., "vector search" ? "FAISS", "LLM fine-tuning" ? "LoRA").
- Multi-signal fusion prevents a single keyword hit from dominating; a keyword-only "HR Manager" with ChatGPT experiments ranks below a backend engineer with NLP + retrieval depth.
- Redrob behavioral signals differentiate equally skilled profiles by availability and engagement - impossible with BM25 alone.

4. Worked Example - Senior AI Engineer JD

JD excerpt: 5-9 years; own ranking/retrieval/matching; embeddings, hybrid retrieval, LLM re-ranking; ship fast, eval frameworks.

Top 3 ranked candidates (from output/ranked_candidates.csv)

Rank | ID | Score | Title / Experience | Explanation (abbrev.)

1 | CAND_0002025 | 0.7844 | Senior AI Engineer, 5.9 yrs | Matches: nlp, faiss \ | Skills include Sentence Transformers, Haystack, Weaviate

2 | CAND_0081686 | 0.7571 | Search Engineer, 6.0 yrs | Matches: embeddings, faiss \ | Skills include ML, Kubeflow, Milvus, BigQuery

3 | CAND_0088025 | 0.7562 | Staff ML Engineer, 8.6 yrs | Matches: nlp, python \ | Skills include RAG, LLMs, QLoRA, Hugging Face

Sanity check: All three sit in the JD's 5-9 year band, carry core AI stack skills (NLP, embeddings, FAISS,

Python, LangChain in top-10), and score high on activity/intent - appropriate for a founding AI engineer role owning retrieval and ranking.

5. Hidden Gems Differentiator

Problem: High-quality candidates with low platform visibility are invisible to keyword search and default sort-by-views.

Formula:

[diagram omitted - see methodology.md]

Surfacing rule: talent_score ? 0.55 AND visibility < 0.35

Interpretation: Strong skill depth and ascending trajectory, but recruiters rarely see them - the "beyond keywords" talent pool.

6. Submission Output Format

Matches Redrob sample_submission.csv:

[diagram omitted - see methodology.md]

Generator: ml_training/02_generate_submission.py

Output path: output/ranked_candidates.csv (top 2,000 by default)

7. Reproducibility

[diagram omitted - see methodology.md]

Environment: Python 3.14 venv with sentence-transformers, faiss-cpu, spacy + en_core_web_sm, scikit-learn, python-docx.

Dataset resolution: backend/src/utils/helpers.py -> resolve_candidates_file() prefers

India_runs_data_and_ai_challenge/candidates.jsonl (100,000 records, ~465 MB).

Optional: FORCE_REINDEX=1 rebuilds the FAISS index with sentence-transformers (recommended if index was previously built with TF-IDF fast path).

8. Limitations (honest)

- Analytics salary charts use synthetic bands for demo UI - not verified compensation data.
- First-pass retrieval uses compact profile text for speed at 100k scale; full-profile encoding is used in enrichment paths.
- Re-indexing 100k profiles with sentence-transformers takes ~20-40 minutes on CPU - persisted index is cached under backend/model_cache/faiss_index/.

CandidateIQ - Beyond Keywords. Real Signals.